The easiest things to look at are these:

- ***Lines of code*** —A large codebase is usually more complicated to build and deploy. This usually also indicates that your codebase does a lot of things, which usually sets your team up for conflicting changes when different developers or teams are working on different features. For better maintainability it's good to keep your modules small and focused.
- ***Duplications*** —The classic problem with duplicate code is that if you change it in one place and not another, you end up with a bug where you didn't change the code. Duplications also directly conflict with coding practices of modularity and reusability (a few more "ilities" that fit under maintainability). Ideally you shouldn't have duplicate code.
- ***Issues*** —These will measure your code against the coding standards for whatever language you're using. The better developers are at writing code aligned with standards, the easier it is to make changes and the easier it is for new developers or other teams to jump in and make changes. Static analysis tools all have classifications of issues from really important to not so important; at a minimum you should ensure that the really important issues are kept at zero.
- ***Complexity*** —Also known as *cyclomatic complexity* or the amount of nested code you have. If you have an `if` statement nested inside a loop and two other `if` statements, you'll have a high cyclomatic complexity. This code is hard to read, debug, and test. Ideally you want to keep complexity as low as possible.

Figure 8.11 shows a screenshot of SonarQube and some default key statistics to be aware of.

Figure 8.11. Key metrics from static analysis that indicate maintainability